

# RouteNet-Fermi: Network Modeling with Graph Neural Networks

Miquel Ferriol-Galmés, Jordi Paillisse, José Suárez-Varela, Krzysztof Rusek, Shihan Xiao, Xiang Shi, Xiang Cheng, Pere Barlet-Ros, and Albert Cabellos-Aparicio

**Abstract**—Network models are an essential block of modern networks. For example, they are widely used in network planning and optimization. However, as networks increase in scale and complexity, some models present limitations, such as the assumption of Markovian traffic in queuing theory models, or the high computational cost of network simulators. Recent advances in machine learning, such as **Graph Neural Networks (GNN)**, are enabling a new generation of network models that are data-driven and can learn complex non-linear behaviors. In this paper, we present RouteNet-Fermi, a custom GNN model that shares the same goals as Queuing Theory, while being considerably more accurate in the presence of realistic traffic models. The proposed model predicts accurately **the delay, jitter, and packet loss of a network**. We have tested RouteNet-Fermi in networks of increasing size (up to **300 nodes**), including samples with mixed traffic profiles — e.g., with complex non-Markovian models — and arbitrary routing and queue scheduling configurations. Our experimental results show that RouteNet-Fermi achieves similar accuracy as computationally-expensive packet-level simulators and scales accurately to larger networks. Our model **produces delay estimates with a mean relative error of 6.24% when applied to a test dataset of 1,000 samples, including network topologies one order of magnitude larger than those seen during training**. Finally, we have also evaluated RouteNet-Fermi with measurements from a physical testbed and packet traces from a real-life network.

**Index Terms**—Network Modeling, Graph Neural Networks, Queuing Theory.

## I. INTRODUCTION

**N**ETWORK modeling is arguably one of the key tools when designing, building, and evaluating computer networks, even since the early days of networking [1]. Network models are used in protocol design, performance evaluation, or network planning, just to cite a few examples. The two most widespread network modeling techniques are analytical models based on Queuing Theory (QT), and packet-level simulators [2], [3].

M. Ferriol, J. Paillisse, J. Suárez-Varela, P. Barlet-Ros and A. Cabellos-Aparicio are with the Barcelona Neural Networking Center, Universitat Politècnica de Catalunya, Spain. K. Rusek is with the AGH University of Science and Technology, Poland. S. Xiao, X. Shi, and X. Cheng are with Huawei Technologies Co., Ltd., China. E-mail: {miquel.ferriol, jordi.paillisse, jose.suarez-varela, pere.barlet, alberto.cabellos}@upc.edu, krzysztof.rusek@agh.edu.pl, {xiaoshihan, shixiang16, chengxiangle1}@huawei.com

This publication is part of the Spanish I+D+i project TRAINER-A (ref.PID2020-118011GB-C21), funded by MCIN/AEI/10.13039/501100011033. This work is also partially funded by the Catalan Institution for Research and Advanced Studies (ICREA) and the Secretariat for Universities and Research of the Ministry of Business and Knowledge of the Government of Catalonia and the European Social Fund.

However, the evolution of computer networks, especially concerning complexity and traffic characteristics, highlights some of the limitations of classical modeling techniques. Despite their tremendous success and widespread usage, some scenarios require more advanced techniques capable of accurately modeling complex traffic characteristics, while scaling to large real-world networks.

Especially, two relevant applications can benefit from advanced network modeling techniques: Network Digital Twins (NDT) [4], and network optimization tools. Commonly, an NDT is referred to as a virtual replica of a physical network that can accurately mimic its behavior and can make performance predictions for any given input condition (e.g., traffic, topology change, or new routing configuration). In other words, an NDT is an accurate network model that can support a wide range of network configurations and that can accurately model the complex non-linear behaviors behind real-world networks. As a result, NDTs can be used to produce accurate performance predictions, carry out what-if analysis, or perform network optimization by pairing it with an optimization algorithm [4], [5].

In the context of network optimization, *we can only optimize what we can model*. Optimization algorithms operate by searching the network configuration space (e.g., to find an alternative routing scheme). For each configuration, a network model is used to estimate the resulting performance to see if it fulfills the optimization goal (e.g., minimize delay [6]). To achieve efficient online optimization, it is essential an accurate and fast network model.

State-of-the-art modeling techniques have important limitations in effectively supporting the stringent requirements of current packet-switched networks. **Queuing Theory imposes strong assumptions on the packet arrival process (Poisson traffic generation), which often is not sufficient to model real-world networks [7]**. Internet traffic has been extensively analyzed in the past two decades [8]–[12] and, despite the community has not agreed on a universal model, **there is consensus that in general aggregated traffic shows strong autocorrelation and a heavy-tail [13]**.

Alternatively, packet-level simulators can accurately model networks. However, this comes at a high computational cost. The cost of a simulator depends linearly on the number of packets forwarded, which can be in the range of millions per second on a single 1Gbps link. In consequence, they are slow and impractical when considering large networks with realistic traffic volumes. This also severely limits its applicability to online network optimization, given the hard time constraints

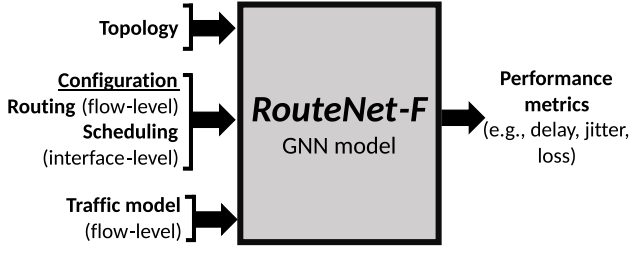


Fig. 1: Black-box representation of RouteNet-F.

of such types of applications.

In this context, **Deep Learning (DL)** offers an extraordinary set of techniques to build accurate data-driven network models. DL models can be trained with real data, without making any assumptions about physical networks. This enables building models with unprecedented accuracy by modeling the entire range of non-linear and multidimensional characteristics.

In this paper, we first make a systematic analysis of the performance of DL techniques for network modeling, using classical discrete-event network simulators as a baseline. Specifically, we analyze the performance of Multilayer Perceptron-based (MLP), Recurrent Neural Network-based (RNN), and Graph Neural Network-based (GNN) models. We find that classical DL techniques, such as MLPs and RNNs, are not practical enough for network modeling as they fail to provide accurate estimates when the network scenario differs from the examples seen during training (e.g., link failure). More recently, GNNs have been proposed as a novel neural network architecture specifically designed to learn over graph-structured data. They have been successfully used in other domains, such as quantum chemistry [14] or logistics [15]. However, in our analysis, we find that standard GNNs [14] do not work well for network modeling and that we need a custom GNN architecture to model computer networks.

As a result, we propose RouteNet-Fermi (RouteNet-F), a GNN architecture for network modeling. RouteNet-F shares the same goals as Queuing Theory. It provides performance estimates (delay, jitter, and packet loss) on given network scenarios (Figure 1) with remarkable accuracy and speed. The proposed model is not limited to Markovian traffic as Queuing Theory; it supports arbitrary models (including autocorrelated processes) which better represent the properties of real-world traffic [13]. Interestingly, it also overcomes one of the main limitations of DL-based models: RouteNet-F generalizes and provides accurate estimates in network scenarios not seen in training (e.g., different topologies, traffic matrices, routing configurations). We benchmark RouteNet-F against a state-of-the-art DL-based model (MimicNet [16]) and with a state-of-the-art queuing theory model. We show that our model outperforms both baselines in all the scenarios, achieving a 5.64% error when tested in a dataset with packet traces coming from a real-world network, an 11% error when evaluated in a physical testbed, and a 6.24% error when estimating the delay on networks over a large dataset with 1,000 network samples, with topologies ranging from 50 to 300 nodes.

As any Deep Learning model [16]–[18], RouteNet-Fermi does not provide strong mathematical performance guarantees.

However, the error of the estimates produced by the model is strongly bounded. The minimum estimated delay assumes no queuing across the path while the maximum assumes that all the queues are full. RouteNet-Fermi will not produce delay estimates outside these bounds.

The implementation of the model used in the evaluation of this paper is publicly available at [19].

## II. CHALLENGES OF DATA-DRIVEN NETWORK MODELING

This section describes the main challenges that data-driven solutions need to address for network modeling. These challenges drove the core design of RouteNet-Fermi.

**Traffic models:** Networks carry different types of traffic, so, supporting arbitrary stochastic traffic models is crucial. Experimental observations show that traffic on the Internet has strong autocorrelation and heavy-tails [13]. In this context, it is well-known that the main limitation of Queuing Theory is that it fails to provide accurate estimates on realistic Markovian models with continuous state space, or non-Markovian traffic models. The challenge for DL-based modeling is: How can we design a neural network architecture that can accurately model realistic traffic models?

**Training and Generalization:** One of the main differences between analytical modeling (e.g., QT) and data-driven modeling is that the latter requires training. In DL, training involves obtaining a *representative* dataset of network measurements. The dataset needs to include a broad spectrum of network operational regimes, ranging from different congestion levels to various routing configurations, among others. In other words, the DL model can predict only scenarios it has previously seen. Note that this is a common property of all neural network architectures.

Ideally, we would obtain this training dataset from a production network, since they commonly have systems in place to measure performance. However, it would be difficult to obtain a *representative* dataset. As we mentioned previously, we would need to measure the production network when it is experiencing extreme performance degradation as the result of link failures, incorrect configurations, severe congestion, etc. However, these situations are not common in production networks, which limits the ability to generate the training dataset. A reasonable alternative is creating these datasets in controlled testbeds, where it is possible to use different traffic models, implement a broad set of configurations, and replicate a wide range of network failures. Thus, the DL model can be trained on samples from this testbed and then, applied to production networks. Hence, the research challenge is: how to design a DL model that can provide accurate estimates in networks not seen during training? This includes topologies, traffic, and configurations (e.g., queue scheduling, routing) different from those seen in the training network testbed.

Leveraging a testbed that is smaller than a production network creates another challenge: the generalization to *larger* networks. Real-world networks include hundreds or thousands of nodes, and building a network testbed at this scale is typically unfeasible. As a result, the DL model should be able to learn from datasets with samples of small network testbeds

and predict metrics for considerably larger networks, e.g., by a factor of 10-100x. Generalizing to larger networks, or graphs in general, is currently an open research challenge in the field of GNNs [20], [21].

**Quality of Service and Scheduling policies:** A key requirement of modern networks is supporting Quality of Service (QoS), usually implemented via scheduling policies and mappings of traffic flows to QoS classes. Hence, a DL model should be able to predict the performance of the input traffic flows with their associated QoS class, similarly to how QT models support a wide range of scheduling policies [22], [23].

### III. LIMITATIONS OF CURRENT NETWORK MODELING TECHNIQUES

This section explores the performance of different DL models with respect to an accurate packet-level simulator and discusses the main limitations of existing network modeling techniques.

#### A. Simulation as a Network Modeling Technique

Network simulators reproduce the network behavior at the granularity of packet events [3], [24]. This way, they can offer excellent accuracy and can be easily extended to include virtually any feature, such as packet scheduling, wireless networks, etc. Some simulators, such as OMNET++ [25] or ns3 [26], are widely used and maintained.

However, their main limitation is the simulation time, especially for networks with high-speed links (10Gbps and above). Hence, depending on the amount of traffic found in the target network, it may become unfeasible to simulate the network [16].

To illustrate this limitation, we simulate different topologies using the OMNET++ simulator to calculate the delay of a set of source-destination flows [CPU Intel Xeon Silver 4210R @ 2.40 GHz]. Network topologies are artificially generated using the Power-Law Out-Degree Algorithm from [27] and a traffic distribution that follows a Poisson process.

Figure 2 shows the simulation time of such networks depending on the number of events. Here, an event refers to a transition in the status of the network (e.g., adding a new packet to a queue). We can see that the simulation time increases linearly and that simulating 4 billion events takes more than 11 hours. Although 4 billion events may appear a large figure, consider that a 10 Gbps link transmitting regular Ethernet frames translates to  $\approx 820k$  events per second or 247 million events in 5 minutes of network activity for a single link. For example, in our experiments, the simulator takes around 8h to compute the performance metrics of a 300-node network.

So, the main limitation of packet-level simulators is the simulation time. On the contrary, packet simulators offer unrivaled accuracy and can simulate virtually any scenario, from different routing configurations to replaying packet traces to simulate unknown traffic models. Because of this, hereafter, we consider the results from the simulator as the ground truth for the evaluations in this paper.

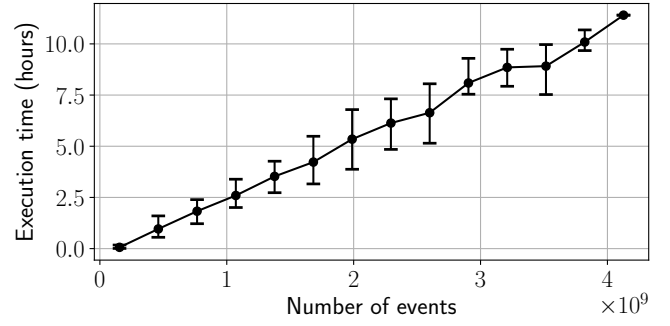


Fig. 2: Simulation time depending on the number of processed events.

#### B. Neural Networks as Network Modeling Techniques

The following sections review the performance of three common Neural Network (NN) architectures in the order of increasing complexity. First, we evaluate the Multilayer Perceptron, one of the simplest NNs. Next, Recurrent Neural Networks which are designed to work with sequences. Finally, we directly input the network into a Graph Neural Network specifically designed to work with graphs. The objective is to create a network model with the NN that can predict performance parameters for input networks with a wide range of characteristics. We are especially interested in the following parameters:

- **Accuracy:** How close is the prediction to simulation values?
- **Different Routing:** Does the accuracy degrade if we change the routing configuration?
- **Link failures:** Quantify if link failures affect the quality of predictions.

We train and test the three neural networks with the same dataset, obtained from simulations with OMNET++. The input values are the network characteristics (topology, routing configuration, traffic model and intensity, etc), and the output values are the delay for each path. Hence, all the errors are computed with respect to the values of the simulator. We use four different datasets:

- **Traffic Models:** In it, we consider traffic models that are non-Poisson, auto-correlated, and with heavy tails. Table IV details the different traffic models.
- **Same Routing:** Where the testing and training datasets contain networks with the *same* routing configurations.
- **Different Routing:** Where the training and testing datasets contain networks with *different* routing configurations.
- **Link failures:** Here, we iteratively remove one link of the topology to replicate a link failure, until we transform the network graph into a connected acyclic graph. This scenario is the most complex since a link failure triggers a change both in the routing and the topology.

To compare the different techniques, we compute the prediction error with respect to the accurate performance values produced by the simulator. Particularly we use the following error metrics: (i) Mean Absolute Percentage Error (MAPE),



TABLE I: Delay prediction using an MLP and an RNN for different traffic models. The error is computed w.r.t. simulation results.

|                 | MLP   |       |       |                | RNN   |       |       |                |
|-----------------|-------|-------|-------|----------------|-------|-------|-------|----------------|
|                 | MAPE  | MSE   | MAE   | R <sup>2</sup> | MAPE  | MSE   | MAE   | R <sup>2</sup> |
| Poisson         | 12.3% | 0.103 | 0.122 | 0.801          | 10.0% | 0.071 | 0.084 | 0.862          |
| Deterministic   | 23.9% | 0.309 | 0.160 | 0.044          | 13.1% | 0.083 | 0.070 | 0.743          |
| On-Off          | 30.4% | 0.438 | 0.240 | 0.002          | 15.2% | 0.065 | 0.082 | 0.851          |
| A. Exponentials | 84.5% | 1.013 | 0.308 | -1.935         | 14.0% | 0.070 | 0.072 | 0.7961         |
| M. Exponentials | 57.1% | 1.058 | 0.363 | -1.679         | 57.8% | 0.528 | 0.457 | -0.338         |
| Mixed           | 41.2% | 0.351 | 0.269 | 0.052          | 17.5% | 0.036 | 0.080 | 0.900          |

(ii) Mean Squared Error (MSE), (iii) Mean Absolute Error (MAE), and (iv) Coefficient of Determination (R<sup>2</sup>).

### C. Multilayer Perceptron

A Multilayer Perceptron (MLP) is a basic kind of NN from the family of feedforward NNs [28]. In short, input data is propagated unidirectionally from the input neuron layer to the output layer. There may be an arbitrary number of hidden layers between these two layers, and this determines how deep is the NN.

1) *Design*: Several works have leveraged an MLP to predict network performance metrics [29]–[31]. Based on this work, we have built an MLP to predict the mean delay for each source-destination pair of nodes of a given network. The MLP has 8,280 inputs and two hidden layers with 4096 neurons and uses Rectified Linear Units (ReLU) as activation functions.

2) *Evaluation*: Table I presents the error when predicting the network delay with respect to the results produced by the network simulator, including several traffic models. We can see that the MLP offers good accuracy for Poisson traffic, but the error increases significantly for the rest of the traffic models showing a MAPE between 23% and 84%.

Likewise, Table II shows the error of predicting the delay for the datasets with the same/different routing and link failures. We can see that the MLP cannot offer an accurate estimate when predicting the delay of a previously unseen routing configuration (1150% of error). This is due to the internal architecture of the MLP. During training, the MLP performs overfitting, meaning that the model *only* learns about the initial network topology used for training and not for any others. When we input a new topology, it does not have sufficient information to make an accurate prediction.

### D. Recurrent Neural Networks

Recurrent Neural Networks (RNN) are a more advanced type of NN. They have shown excellent performance when

TABLE II: Delay prediction using an MLP and an RNN for the same and different routing configurations w.r.t. those seen during training, and considering various link failures. The error is relative to simulation results.

|               | MLP   |       |       |                | RNN   |       |       |                |
|---------------|-------|-------|-------|----------------|-------|-------|-------|----------------|
|               | MAPE  | MSE   | MAE   | R <sup>2</sup> | MAPE  | MSE   | MAE   | R <sup>2</sup> |
| Same Routing  | 12.3% | 0.103 | 0.122 | 0.801          | 10.0% | 0.071 | 0.084 | 0.862          |
| Diff. Routing | 1150% | 28.3  | 2.96  | -40.0          | 30.5% | 0.553 | 0.282 | 0.197          |
| Link Failures | 125%  | 3.69  | 1.03  | -0.191         | 63.8% | 2.971 | 0.870 | 0.0417         |

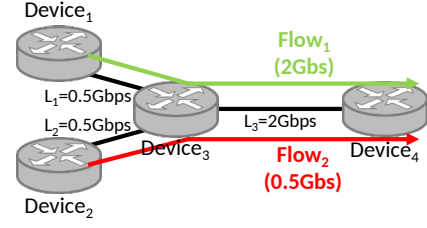


Fig. 3: Sample Topology with 4 nodes, three links, and two flows.

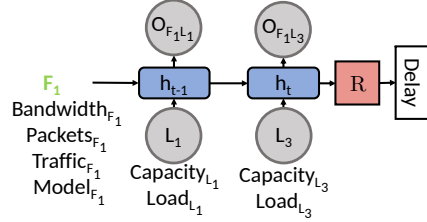


Fig. 4: Recurrent Neural Network model for the Sample Topology (Figure 3)

processing sequential data [32]. This is mainly because they connect some layers to the previous ones, which gives them the ability to keep the state along sequences.

1) *Design*: Several works [33]–[35] propose RNNs as a way to predict network performance. In this experiment, we build a sequential model with an RNN (Figure 4). Particularly, we choose a Gated Recurrent Unit (GRU).

We initialize the state of each path with the sequence of nodes in the path and the features of the traffic model (e.g., packets, bandwidth,  $\lambda$ ,  $\epsilon$ ,  $\alpha$ , or on-off time), and we update the state of each link across the path. As an example, Figure 4 shows the structure of an RNN to model the sample network from Figure 3. We can see that the path of *flow*<sub>1</sub> is composed of *L*<sub>1</sub> and *L*<sub>3</sub>. Once the path state has been computed, an MLP with 2 hidden layers computes the final output.

2) *Evaluation*: We train the RNN with the same datasets as the previous subsection. Although the RNN supports better different traffic models than the MLP (Table I), it still struggles to produce accurate predictions when there are routing or topology changes (Table II), especially for different routing configurations (30% error), or when removing links (63%).

The reason behind the lack of capability of RNNs to understand routing changes and link failures is due to its internal architecture. RNNs can accommodate different end-to-end paths in the network (i.e., series of routers and links), thereby, making it easier to perform predictions for paths never seen in the training phase. However, this structure cannot store and update the status of individual links in the topology due to the inter-dependency between links and traffic flows (i.e., routing). In other words, if the status of a link changes, it affects several flows, and vice-versa. This generates circular dependencies that RNNs are not able to model (see more details in Sec. IV).

### E. Graph Neural Networks

Networks are fundamentally represented as graphs, where networking devices are the graph nodes and the links connecting devices are the graph edges. This interconnection translates to the fact that the different elements in the network are dependent on each other. Since most standard DL models (e.g., MLP, RNN) assume independent flow-level data points. This renders them inaccurate for our use case. Hence, a model that is capable of processing directly the network graph is arguably more desirable for network modeling, because it will be able, not only to obtain information from the individual nodes and edges but also from the underlying data structure (i.e., the relationships between the different elements) [36].

GNNs [37] are a type of neural network designed to work with graph-structured data. They have two key characteristics that make them a good candidate for a network model. First, GNNs have the ability to store node-level hidden states and update them in each iteration. Second, they process the input data directly as a graph, both during training and inference. This means that the internal structure of the neural network depends on the input graph. Hence, they can dynamically adapt to the underlying dependencies between the different elements of a network [38], [39]. The latter is of special importance, since changes in the network graph (e.g., routing modifications, link failures) are the main limitation of other DL-based models, such as MLPs and RNNs, as we have seen in the previous subsections.

In this section, we build a standard GNN model to predict the mean per-flow delay in networks.

1) *Design*: We implement a Message-Passing Neural Network (MPNN), a powerful state-of-the-art GNN architecture that can efficiently capture dependencies between the elements of input graphs [14]. We define a graph  $G$  described as a set of vertices (or nodes)  $E$  and a set of edges (or links)  $V$ . Each node has a set of features  $x_v$ , and edges also have some features  $e_{vw}$ . The execution of an MPNN can be divided into three phases, an initialization phase, a message-passing phase, and a readout phase. The first one defines a hidden state ( $h_v^0$ ) using the node features ( $x_v$ ). The second one is an iterative process that runs for  $T$  time steps and that is defined by two functions: the message function  $M_t$  and the update function  $U_t$ . During this phase, node hidden states ( $h_v^t$ ), represented as fixed-size vectors, attempt to encode some *meaningful* information, and are iteratively updated by exchanging messages  $m_v^{t+1}$  with their neighbors:

$$m_v^{t+1} = \sum_{w \in N(v)} M_t(h_v^t, h_w^t, e_{vw}) \quad (1)$$

$$h_v^{t+1} = U_t(h_v^t, m_v^{t+1}) \quad (2)$$

where  $N(v)$  represents the neighbors of  $v$  in graph  $G$ . Finally, the readout phase computes the output vector using a readout function  $R$  that takes as input the final hidden states  $h_v^T$ :

$$\hat{y} = R(\{h_v^T | v \in G\}) \quad (3)$$

In our case, the input of the MPNN is the network topology graph. It performs  $T=4$  message-passing iterations and the hidden state dimension is 32. The readout function is implemented with a two-layer fully connected NN with ReLUs as activation functions for the hidden layers and a linear activation for the output layer.

2) *Evaluation*: We evaluate the accuracy of the MPNN model when predicting the mean flow-level delay, as in previous subsections. Table III presents the delay prediction errors in the same scenarios of the previous experiments: routing configurations, both seen and not seen during training, and link failures. Unfortunately, the results are similar to those of the RNN: the routing configurations from the training dataset are easy to predict, with an error as low as 3%, while new routing configurations and link failures increase the error significantly (respectively, 50% and 125% of MAPE), thus showing even larger errors than the RNN model.

The main reason behind the poor accuracy of the MPNN model is that the architecture of this model is directly built based on the network topology without taking into account the paths traversed by different traffic flows (i.e., the routing configuration), which is a fundamental property to understand inter-dependencies between flows and links. More specifically, when we test the model with the same routing configuration, it learns the relationships between flows and links. However, when we change this configuration, those previously-learned relationships are no longer valid, and the model is not able to capture the relationships between elements in the new scenario.

This intuition is better understood with an extreme packet loss example. Let's suppose we have the sample network in Figure 3. The first flow (Flow<sub>1</sub>) is transmitting at a rate of 2Gbps, and the second one (Flow<sub>2</sub>) is transmitting at a rate of 0.5Gbps. As we can see, L<sub>1</sub> has a maximum capacity of 0.5Gbps. Because of this, Flow<sub>1</sub> will experience a large packet loss, which causes the traffic of Flow<sub>1</sub> at L<sub>3</sub> to be at most 0.5Gbps. Hence, instead of having 2Gbps+0.5Gbps of aggregated traffic at L<sub>3</sub>, it will only have 1Gbps. Now, Flow<sub>2</sub>, which initially could have experienced a lot of network congestion when going through the 2Gbps link will experience none as the state of the link changed.

Knowing this, we can see how there is a circular dependency between the flows and links found in the network. At the same time, the state of a flow depends on the state of the links they traverse, and the state of the links depends on the state of the flows passing through them.

If we apply a standard GNN over this example, the state of each flow is not updated at each hop. Therefore, the GNN does

TABLE III: Delay prediction using an MPNN for the same and different routing configurations w.r.t. those seen during training, and considering various link failures. The error is relative to simulation results.

|               | MAPE  | MSE   | MAE   | R <sup>2</sup> |
|---------------|-------|-------|-------|----------------|
| Same Routing  | 3.0%  | 0.002 | 0.016 | 0.994          |
| Diff. Routing | 50.0% | 0.609 | 0.307 | 0.115          |
| Link Failures | 82.2% | 3.41  | 0.949 | -0.099         |

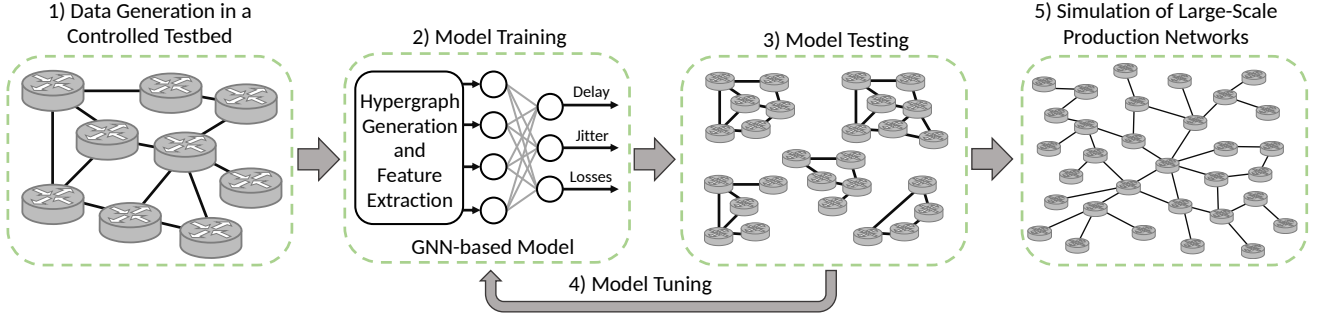


Fig. 5: End-to-end workflow of RouteNet-F. 1) Collection of small-scale observations coming from a controlled environment, 2) Model training, 3) Model testing with various configurations (e.g., routing, scheduling) never seen during the training phase, 4) hyper-parameter tuning to balance highly accurate predictions with performance, 5) Simulation of large-scale production networks. One of the main advantages of RouteNet-F is that usually time-consuming steps like 1), 2), 3), and 4) are all done at small scales and, therefore, are fast as well.

not have a structure that represents how the delay depends on both the links (topology) and the flows that go through each specific router.

Hence, we conclude that feeding the MPNN directly with the network topology graph is not sufficient to accurately perform network modeling. However, more complex and customized GNN-based architectures can still be powerful for modeling the inter-dependencies between the different network elements and generalizing over new network scenarios by exploiting the underlying graph structure.

#### IV. ROUTENET-FERMI

This section describes the internal GNN architecture of RouteNet-Fermi (hereafter referred to as RouteNet-F). This GNN-based model implements a custom three-stage message-passing algorithm that represents key elements for network modeling (e.g., topology, queues, traffic flows). RouteNet-F supports a wide variety of features present in real-world networks, such as multi-queue QoS scheduling policies or complex traffic models.

Figure 1 shows a black-box representation of RouteNet-F. The input of this model is a network sample, defined by: a network topology, a routing scheme (flow level), a queuing configuration (interface level), and a set of traffic flows characterized by some parameters. As output, the model produces estimates of relevant performance metrics at a flow-level granularity (e.g., delay, jitter, packet loss). Figure 5 shows the end-to-end workflow to train, validate, and use RouteNet-Fermi.

##### A. Model Description

RouteNet-F is based on two main design principles: (i) *finding a good representation* of the network components supported by the model (e.g., traffic models, routing, queue scheduling), and (ii) *exploit scale-independent features* of networks to accurately scale to larger networks unseen during training. These two aspects are further discussed in the next two subsections.

##### B. Representing network components and their relationships

First, let us define a network as a set of source-destination flows  $\mathcal{F} = \{f_i : i \in (1, \dots, n_f)\}$ , a set of queues on  $\mathcal{Q} = \{q_j : j \in (1, \dots, n_q)\}$ , and a set of links  $\mathcal{L} = \{l_k : k \in (1, \dots, n_l)\}$ . According to the routing configuration, flows follow a source-destination path. Hence, we define flows as sequences of tuples with the queues and links they traverse  $f_i = \{(q^{i,1}, l^{i,1}), \dots, (q^{i,M}, l^{i,M})\}$ , where  $M$  is the path length of the flow (number of links). Let us also define  $Q_f(q_j)$  and  $L_f(l_k)$  as functions that respectively return all the flows passing through a queue  $q_j$  or a link  $l_k$ . Also,  $L_q(l_k)$  is defined as a function that returns the queues  $q_{l_k} \in \mathcal{Q}$  injecting traffic into link  $l_k$  (i.e., the queues at the output port to which the link is connected).

Following the previous notation, RouteNet-F considers an input graph with three main components: (i) the physical links  $\mathcal{L}$  that shape the network topology, (ii) the queues  $\mathcal{Q}$  at each output port of network devices, and (iii) the active flows  $\mathcal{F}$  in the network, which follow some specific src-dst path (i.e., sequences of queues and links). Traffic in flows is generated from a given traffic model. From this, we can extract three basic principles:

- 1) The state of flows (e.g., delay, throughput, packet loss) is affected by the state of the queues and links they traverse (e.g., queue/link utilization).
- 2) The state of queues (e.g., occupation) depends on the state of the flows passing through them (e.g., traffic volume, burstiness).
- 3) The state of links (e.g., utilization) depends on the states of the queues that can potentially inject traffic into the link, and the queue scheduling policy applied over these queues (e.g., Strict Priority, Weighted Fair Queuing).

Formally, these principles can be formulated as follows:

$$\mathbf{h}_{f_i} = G_f(\mathbf{h}_{q^{i,1}}, \mathbf{h}_{l^{i,1}}, \dots, \mathbf{h}_{q^{i,M}}, \mathbf{h}_{l^{i,M}}) \quad (4)$$

$$\mathbf{h}_{q_j} = G_q(\mathbf{h}_{f_1}, \dots, \mathbf{h}_{f_i}), \quad f_i \in Q_f(q_j) \quad (5)$$

$$\mathbf{h}_{l_k} = G_l(\mathbf{h}_{q_1}, \dots, \mathbf{h}_{q_J}), \quad q_j \in L_q(l_j) \quad (6)$$

Where  $G_f$ ,  $G_q$ , and  $G_l$  are some unknown functions, and  $\mathbf{h}_f$ ,  $\mathbf{h}_q$  and  $\mathbf{h}_l$  are latent variables that encode information

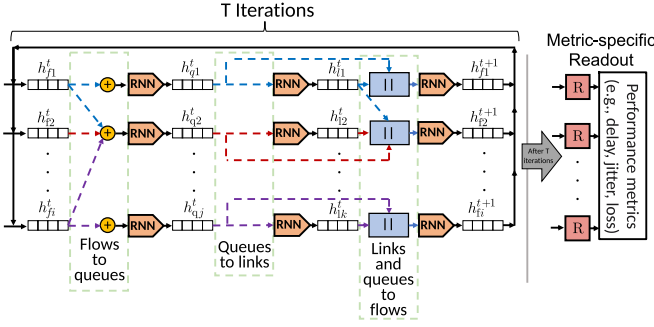


Fig. 6: Schematic representation of RouteNet-F.

about the state of flows  $\mathcal{F}$ , queues  $\mathcal{Q}$ , and links  $\mathcal{L}$  respectively. Note that these principles define a circular dependency between the three network components ( $\mathcal{F}$ ,  $\mathcal{Q}$ , and  $\mathcal{L}$ ) that must be solved to find latent representations satisfying the equations above.

To solve the circular dependencies defined in Equations (4)-(6), RouteNet-F implements a three-stage message passing algorithm that combines the states of flows  $\mathcal{F}$ , queues  $\mathcal{Q}$ , and links  $\mathcal{L}$ , and updates them iteratively. Finally, it combines these states to estimate flow-level delays, jitters, and packet loss. Figure 6 shows a schematic representation of the internal three-stage message-passing architecture of this model.

Algorithm 1 describes the architecture of RouteNet-F. First, hidden states  $h_f$ ,  $h_q$ , and  $h_l$  are initialized using the functions  $HS_f$ ,  $HS_q$ , and  $HS_l$  respectively (lines 1-3). These functions encode the initial features  $x_f$ ,  $x_q$ , and  $x_l$  into fixed-size vectors that represent feature embeddings. The initial features of flows  $x_f$  are defined as an  $n$ -element vector that characterizes the flow's traffic. For example, in our case, this vector includes the average traffic volume transmitted in the flow  $\lambda$ , and some specific parameters of the traffic model, such as  $t_{on}$  and  $t_{off}$  for On-Off traffic distributions or  $\alpha$  and  $\beta$  for exponential models. We set the initial features of links  $x_l$  as (i) the link load  $x_{l_{load}}$ , and (ii) the scheduling policy at the output port of the link (FIFO, Strict Priority, Weighted Fair Queuing, or Deficit Round Robin). For the scheduling policy, we use a one-hot encoding. The calculation of the link load  $x_{l_{load}}$  is defined in more detail later (Sec. IV-C). Lastly, the initial features of queues  $x_q$  include: (i) the buffer size, (ii) the queue order/priority level (one-hot encoding), and (iii) the weight (only for Weighted Fair Queuing or Deficit Round Robin configurations).

Once all the hidden states are initialized, the message-passing phase starts. This phase is executed for  $T$  iterations (loop from line 4), where  $T$  is a configurable parameter of the model. Each message passing iteration is divided into three stages, which represent respectively the message exchanges and updates of the hidden states of flows  $h_f$  (lines 5-10), queues  $h_q$  (lines 11-14), and links  $h_l$  (lines 15-19).

Finally, the loop from line 20 computes the different flow-level performance metrics. Here, function  $R_{fd}$  (line 24) and  $R_{fj}$  (line 28) represent a readout function that is individually applied to the hidden states of flows as they pass through

#### Algorithm 1 Internal architecture of RouteNet-F.

**Input:**  $\mathcal{F}$ ,  $\mathcal{Q}$ ,  $\mathcal{L}$ ,  $x_f$ ,  $x_q$ ,  $x_l$

**Output:**  $\hat{y}_{fd}$

```

1: for each  $f \in \mathcal{F}$  do  $h_f^0 \leftarrow HS_f(x_f)$ 
2: for each  $q \in \mathcal{Q}$  do  $h_q^0 \leftarrow HS_q(x_q)$ 
3: for each  $l \in \mathcal{L}$  do  $h_l^0 \leftarrow HS_l(x_l)$ 

4: for  $t = 0$  to  $T-1$  do                                     ▶ Message Passing Phase
5:   for each  $f \in \mathcal{F}$  do                                       ▶ Message Passing on Flows
6:      $\Theta([\cdot, \cdot]) \leftarrow FRNN(h_f^t, [\cdot, \cdot])$                  ▶ FRNN Initialization
7:     for each  $(q, l) \in f$  do
8:        $h_{f,l}^t \leftarrow \Theta([h_q^t, h_l^t])$                      ▶ Flow: Aggr. and Update
9:        $\tilde{m}_{f,q}^{t+1} \leftarrow h_{f,l}^t$                          ▶ Flow: Message Generation
10:       $h_f^{t+1} \leftarrow h_{f,l}^t$ 
11:   for each  $q \in \mathcal{Q}$  do                                       ▶ Message Passing on Queues
12:      $M_q^{t+1} \leftarrow \sum_{f \in \mathcal{Q}_f(q)} \tilde{m}_{f,q}^{t+1}$              ▶ Queue: Aggregation
13:      $h_q^{t+1} \leftarrow U_q(h_q^t, M_q^{t+1})$                  ▶ Queue: Update
14:      $\tilde{m}_q^{t+1} \leftarrow h_q^{t+1}$                              ▶ Queue: Message Generation
15:   for each  $l \in \mathcal{L}$  do                                       ▶ Message Passing on Links
16:      $\Psi(\cdot) \leftarrow LRNN(h_l^t, \cdot)$                        ▶ LRNN Initialization
17:     for each  $q \in \mathcal{L}_q(l)$  do
18:        $h_l^t \leftarrow \Psi(\tilde{m}_q^{t+1})$                          ▶ Link: Aggr. and Update
19:      $h_l^{t+1} \leftarrow h_l^t$ 

20: for each  $f \in \mathcal{F}$  do                                       ▶ Flow: Readout
21:    $\hat{y}_{fd} = 0$                                                ▶ Initializing the flow delay
22:    $\hat{y}_{fj} = 0$                                                ▶ Initializing the flow jitter
23:   for each  $(q, l) \in f$  do
24:      $\hat{d}_q = R_{fd}(h_{f,l}^T)/x_{lc}$                                ▶ Queuing delay
25:      $\hat{d}_l = x_{f_{ps}}/x_{lc}$                                      ▶ Transmission delay
26:      $\hat{d}_{link} = \hat{d}_q + \hat{d}_l$ 
27:      $\hat{y}_{fd} = \hat{y}_{fd} + \hat{d}_{link}$                                ▶ Sum of link delays along the flow
28:      $\hat{y}_{fj} = \hat{y}_{fj} + R_{fj}(h_{f,l}^T)/x_{lc}$                  ▶ Sum of link jitters along the flow
29:    $\hat{y}_{fl} = R_{fl}(h_f^T)$                                      ▶ Packet loss prediction

```

a specific link ( $h_{f,l}$ ). The output of these functions is the average queue occupancy and the delay variation (i.e., jitter) seen by the flow at that link. Note that different flows may experience different queue occupancies and jitter depending on their traffic properties (e.g., traffic volume, burstiness). Lastly, these link-level delay and jitter estimates are combined to compute the final flow-level delay  $\hat{y}_{fd}$  and jitter  $\hat{y}_{fj}$ . This calculation is further described in Section IV-C. Similarly,  $R_{fl}$  (line 29) is applied to the hidden states of flows  $h_f$  to compute the per-flow packet loss rate.

#### C. Scaling to larger networks: scale-independent features

Data-driven models typically need to see edge cases that are uncommon in real-world production networks (e.g., link failures). This means that collecting data directly from production networks requires testing configurations that might break the correct operation of the network. As a result, data-driven network models should be typically trained with data from controlled network testbeds. However, network testbeds are usually much smaller than real networks. In this context, it is essential for our model to effectively scale to larger networks than those seen during the training phase.

It is well-known that GNN models have an unprecedented capability to generalize over graph-structured data [38], [39]. In the context of scaling to larger graphs, it is also known that GNNs keep good generalization capabilities as long as the spectral properties of graphs are similar to those seen during training [40]. In our particular case, the internal message passing architecture of RouteNet-F generalizes accurately to graphs with similar structures (e.g., a similar number of queues at output ports, or a similar number of flows aggregated in queues). In practice, this means that RouteNet-F should be able to generalize to larger topologies when trained with smaller ones, as long as the networks used for training had the same spectral properties as the larger ones. More details about this can be found in Section V, which presents an empirical evaluation of RouteNet-F's generalization capabilities).

However, scaling to larger networks often entails other aspects beyond the topology size. Two key elements require special attention: link capacities and the range of output variables. First, larger networks naturally have larger link capacities. This in turn results in larger traffic aggregates on core links of the network. Such traffic intensities may fall in ranges not seen in smaller topologies. Second, as the network scales, it inherently has longer end-to-end paths, which result in increased end-to-end path delays. Again, some of these delays may fall outside the range of delays used when training with smaller topologies. These out-of-range parameters require devising mechanisms to effectively scale on them.

1) *Scaling to larger link capacities:* In RouteNet-F (Algorithm 1), the most straightforward way to represent the link capacity  $x_{l_c}$  would be as an initial feature of the links' hidden states  $x_l$ . However, the fact that  $x_{l_c}$  would be encoded as a numerical input value would then introduce inherent limitations to scale to larger capacity values. Indeed, scaling to out-of-distribution numerical values is recognized as a generalized limiting factor among all neural network models [41], [42].

Our approach is to exploit particularities from the network domain to find scale-independent representations for link capacities. These representations define link capacities and how they relate to other link-level features that impact performance (e.g., the aggregated traffic in the link), so they can be used to accurately estimate performance metrics (e.g., delay, jitter, packet loss). Inspired by traditional QT methods, we aim to encode in RouteNet-F the relative ratio between the arrival rates on links (based on the traffic aggregated in the link) and the service times (based on the link capacity). This enables the possibility to infer the output performance metrics of our model from scale-independent values. As a result, instead of directly using the numerical link capacity values, we introduce the *link load*  $x_{load}$  in the initial feature vector of links  $x_l$ . Particularly, we compute the link load as follows:

$$x_{load} = \frac{1}{x_{l_c}} \sum_{f \in L_f(l_j)} \lambda_f \quad (7)$$

Where  $\lambda_f$  is the average traffic volume of the flows that traverse the link  $l_j$ , and  $x_{l_c}$  is the link capacity. In other words, we compute the link load as the summation of all the traffic that would traverse the link without considering possible losses

and divide it by the link capacity. Then, through the iterative message-passing process, the GNN model updates the load values after estimating the packet loss.

2) *Different output ranges:* The previous mechanism enables us to keep scale-independent features along with the message-passing phase of our model (loop lines 4-19 in Alg. 1), while it is still needed to extend the scale independence to the output layer of the model. Note that in larger networks, delay values can vary with respect to those seen during the training in smaller networks. This is because flows can go through links with higher capacities, or because flows can potentially traverse longer paths. This again poses the challenge of generalizing to ranges of delays not seen in the training phase. Equivalently, this also applies to the prediction of flow jitter and packet loss.

To overcome this potential limitation, RouteNet-F infers delays indirectly from the mean queue occupancy on forwarding devices. Based on traditional QT models, RouteNet-F infers the flow delay as a linear combination of the estimated queuing delays (line 24) and the transmission delays after crossing a link (line 25). Note that a potential advantage with respect to traditional QT models is that the queue occupancy estimates produced by RouteNet-F can be more accurate, especially for autocorrelated and heavy-tail traffic models.

We call the values produced by the  $R_{fd}$  function the *effective queue occupancy*, which is defined as the mean queue occupancy experienced by a given flow  $f_i$  as it passes through a specific forwarding device. More precisely, this value is the average number of bits that have to be served on a specific output port before the packets of flow  $f_i$  are transmitted. As an example, let us consider the case of packets from a flow with low priority, which are mapped to low-priority queues. If forwarding devices implement a multi-queue Strict Priority scheduling policy, the effective queue occupancy seen by those low-priority packets should include all the bits to be served in the queues with higher priority.

The prediction of this effective queue occupancy — instead of directly predicting delays — helps overcome the practical limitation of producing out-of-range delay values with the readout function  $R_{fd}$ . In this case, the values produced by  $R_{fd}$  are bounded between 0 and the maximum buffer size at the output ports of forwarding devices. Note that the buffer size is a device-specific feature that is independent of the network size.

Lastly, RouteNet-F produces flow-level delay predictions  $\hat{y}_{fd}$  by combining the estimated queuing and transmission delays. The queuing delay  $\hat{d}_q$  is indirectly estimated by using the effective queue occupancies (in bits) on queues for a particular flow. Particularly, queue occupancy values are estimated by the readout function  $R_{fd}(\mathbf{h}_{f,l}^T)$ . Then, they are divided by the capacity of the link connected to the output port  $x_{l_c}$  to eventually produce a queuing delay estimate  $\hat{d}_q$ . Likewise, the transmission delay  $\hat{d}_t$  is computed by dividing the mean flow packet size  $x_{fps}$  by the link capacity  $x_{l_c}$ . With this, RouteNet-F estimates the delay of a flow after passing through a specific forwarding device and a link ( $\hat{d}_{link}$ ):



$$\hat{d}_q = \frac{R_{fd}(\mathbf{h}_{f,l}^T)}{x_{lc}} \quad (8)$$

$$\hat{d}_t = \frac{x_{fps}}{x_{lc}} \quad (9)$$

$$\hat{d}_{link} = \hat{d}_q + \hat{d}_t \quad (10)$$

Hence, we can compute end-to-end flow delays as the sum of all the link delays  $\hat{d}_{link}$  along the flow (loop lines 20-27 in Algorithm 1). Note that the function responsible for computing the effective queue occupancy,  $R_{fd}(\mathbf{h}_{f,l}^T)$  takes as input the hidden state of the flow at a specific link  $\mathbf{h}_{f,l}^T$ , instead of directly considering queue states  $\mathbf{h}_q^T$ . This is because each flow can experience a different queuing behavior depending on its properties (e.g., traffic burstiness).

Likewise, jitter estimates  $\hat{y}_{fj}$  are produced by combining the jitter predictions of all links along the flow. These predictions are made by the  $R_{fj}$  function, which takes as input the hidden state of the flow at a specific link  $\mathbf{h}_{f,l}^T$ . Note that we define the jitter as the relative fluctuation with respect to the mean delay, that is, the ratio between the delay variance divided by the flow mean delay.

In the case of packet loss, RouteNet-F makes predictions directly on flows' hidden states  $\mathbf{h}_f^T$ . We define the packet loss as the relative ratio of packets dropped with respect to the packets transmitted by the source; hence it is a bounded value  $\hat{y}_{fi} \in [0, 1]$ . We estimate it with the  $R_{fi}$  function (line 29 in Algorithm 1).

#### D. Training and Implementation

We implement RouteNet-Fermin in TensorFlow and it is publicly available at [19].

As in any other ML model, fine-tuning the hyperparameters of RouteNet-F is crucial for achieving optimal performance and accuracy. Two key parameters to consider are the hidden state vectors ( $\mathbf{h}_f$ ,  $\mathbf{h}_q$ ,  $\mathbf{h}_l$ ) and the number of message passing iterations ( $T$ ). The size of the hidden state vectors determines how much information the model can encode, with larger sizes allowing for more information but also harming performance. Similarly, a larger number of message-passing iterations can help the model reach a higher level of convergence, but at the cost of increased computation. Through grid search experiments, we selected a set of hyperparameters that provide good accuracy while also being efficient. Specifically, we set the size of all the hidden state vectors to 32 elements and  $T$  to 8 message-passing iterations.

We implement the functions *FRNN* (Flow-Level RNN), *LRNN* (Link-Level RNN), and  $U_q$  (Queue Update Function) as Gated Recurrent Units (GRU). Functions  $HS_f$ ,  $HS_q$ , and  $HS_l$  are implemented as 2-layer fully-connected neural networks with ReLU activation functions with 32 units each. Similarly, functions  $R_{fd}$ ,  $R_{fj}$ , and  $R_{fi}$  are implemented as a 3-layer fully-connected neural networks with ReLU activation function for the hidden layers, and a linear one for the output layer (except for  $R_{fi}$  that uses a Sigmoid activation function). Note that, the whole architecture of RouteNet-F (Algorithm 1) constitutes a fully-differentiable function. This means that it

can be trained end-to-end using as input the network samples and as output the different flow-level performance metrics (e.g., delay, jitter, packet loss) as illustrated in the black box diagram of Figure 1.

For each set of experiments, we train RouteNet-F during 150 epochs of 2,000 samples each. We set as loss function the Mean Absolute Percentage Error for the delay experiments, the Mean Squared Error for jitter, and the Mean Absolute Error for the packet loss. In all the cases, we use an Adam optimizer with an initial learning rate of 0.001.

## V. EVALUATION

In this section, we evaluate the performance of RouteNet-F in a wide range of relevant scenarios. First, we evaluate RouteNet-F in a variety of scenarios with complex traffic models and scheduling policies and compare it to a state-of-the-art Queuing Theory (QT) benchmark from [43]. In it, the network is modeled as a  $M/M/1/b$  system where each queue along a path is treated independently. Note that the baseline, like the majority of QT models, assumes that arrivals to each queue are approximated by a Poisson process and that service times are exponentially distributed. Under these assumptions, the model derives analytical results for queue throughput, delay distributions, and blocking probabilities. A public implementation of the QT model can be found at [44].

Then, we evaluate RouteNet-F's generalization capabilities when evaluated in topologies x30 times larger than the ones seen during training and compare its inference times. Finally, we benchmark RouteNet-F in real-world scenarios with data from a real testbed, with traffic coming from real-world networks, and compare its performance with MimicNet [16] a state-of-the-art DL-based model.

### A. Performance Analysis

1) *Methodology*: In all the experiments (except for subsection V-C1 that comes from a real testbed), the ground truth is obtained using a packet-level simulator (Section V-A2). Unless specified, in each evaluation we perform 50k experiments with a random configuration (src-dst routing, traffic intensity, per-interface scheduling policy, queue size, and traffic model), and compute the mean average delay, jitter, and packet loss. Then we compute the error of RouteNet-F's and QT's estimates with respect to the results of the packet simulator. For a fair comparison, the evaluation samples have not been used in the training phase of RouteNet-F.

2) *Dataset*: We generate our dataset by simulating a wide range of network scenarios with the OMNet++ network simulator, v5.5.1 [25]. An image of the simulator is publicly available and can be found at [45]. Each dataset sample corresponds to a single simulation, and we record the mean delay, jitter, and packet loss for all the flows in the network, as well as queue-level statistics (e.g., mean occupancy, average packet loss, or average packet size). We select the different scenarios to simulate by randomly sampling from the possible values of all the input variables (Table IV). The traffic models *autocorrelated exponentials* and *modulated exponentials* reproduce realistic Internet traffic [9], [43]. We define the

traffic intensity ( $TI$ ) as a tunable parameter that defines the overall traffic load in the network scenario.  $TI$  represents how congested is the network. In our dataset, it ranges from 400 to 2000 bits per time unit, with 400 being the lowest congested network (0% avg. packet loss) and 2000 a highly congested network ( $\approx 3\%$  avg. packet loss).

3) *Traffic Models*: This section analyzes the accuracy of RouteNet-F in a wide range of traffic models. The experiment is organized such that, for each traffic model, we add a degree of complexity by changing its first and second-order statistics (i.e., variance and autocorrelation). We start with simple traffic models such as Poisson or Constant Bitrate and end by testing more complex models that are better approximations of traffic seen in Internet links.

Tables V and VI show the errors of the delay and jitter for both, RouteNet-F and QT, with respect to the values obtained using the simulator. We can see that RouteNet-F achieves excellent accuracy results, producing very accurate estimates of delay and jitter in all traffic models: the worst cases are 5.21% and 10.40% for delay and jitter, respectively.

TABLE IV: Simulation variables.

|                       |                                                                                                                                                                    |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Topology              | NSFNET [46], GEANT [47], GBN [48], and scale-free synthetic topologies following the Power-Law Out-Degree algorithm [27].                                          |
| Traffic Model         | 6 options: Poisson, On-Off, Constant Bitrate, Autocorrelated Exponentials, Modulated Exponentials (according to [43]), and all models mixed.                       |
| Traffic Intensity     | Random traffic intensities to generate packet loss between 0% and 3%.                                                                                              |
| Queuing Configuration | 1, 2, or 3 queues per port.<br>Queue size: 8, 16, 32, or 64 kbits.<br>Policy: First In First Out, Strict Priority, Weighted Fair Queuing, and Deficit Round Robin. |

TABLE V: Delay prediction using the QT baseline and RouteNet-F for different traffic models. The error is computed w.r.t. simulation results.

|                 | QT    |              |              |                | RouteNet-F   |              |              |                |
|-----------------|-------|--------------|--------------|----------------|--------------|--------------|--------------|----------------|
|                 | MAPE  | MSE          | MAE          | R <sup>2</sup> | MAPE         | MSE          | MAE          | R <sup>2</sup> |
| Poisson         | 12.6% | <b>0.001</b> | <b>0.017</b> | 0.998          | 2.1%         | <b>0.001</b> | <b>0.017</b> | <b>0.999</b>   |
| Deterministic   | 22.4% | 0.715        | 0.321        | 0.611          | <b>4.43%</b> | <b>0.029</b> | <b>0.048</b> | <b>0.984</b>   |
| On-Off          | 23.1% | 0.784        | 0.363        | 0.613          | <b>2.90%</b> | <b>0.009</b> | <b>0.035</b> | <b>0.995</b>   |
| A. Exponentials | 21.1% | 0.686        | 0.316        | 0.618          | <b>2.62%</b> | <b>0.010</b> | <b>0.030</b> | <b>0.994</b>   |
| M. Exponentials | 68.1% | 1.10         | 0.798        | 0.145          | <b>5.21%</b> | <b>0.013</b> | <b>0.061</b> | <b>0.989</b>   |
| Mixed           | 35.1% | 0.721        | 0.430        | 0.560          | <b>4.71%</b> | <b>0.018</b> | <b>0.054</b> | <b>0.988</b>   |

TABLE VI: Jitter prediction using the QT baseline and RouteNet-F for different traffic models. The error is computed w.r.t. simulation results.

|                 | QT    |       |       |                | RouteNet-F   |              |              |                |
|-----------------|-------|-------|-------|----------------|--------------|--------------|--------------|----------------|
|                 | MAPE  | MSE   | MAE   | R <sup>2</sup> | MAPE         | MSE          | MAE          | R <sup>2</sup> |
| Poisson         | 71.9% | 0.013 | 0.072 | 0.849          | <b>6.26%</b> | <b>0.001</b> | <b>0.013</b> | <b>0.980</b>   |
| Deterministic   | 99.0% | 0.057 | 0.067 | -1.86          | <b>7.17%</b> | <b>0.001</b> | <b>0.008</b> | <b>0.924</b>   |
| On-Off          | 69.4% | 0.057 | 0.098 | 0.425          | <b>8.50%</b> | <b>0.004</b> | <b>0.018</b> | <b>0.959</b>   |
| A. Exponentials | 74.3% | 0.025 | 0.067 | 0.246          | <b>6.29%</b> | <b>0.001</b> | <b>0.008</b> | <b>0.973</b>   |
| M. Exponentials | 91.4% | 1.34  | 0.834 | -0.622         | <b>10.3%</b> | <b>0.036</b> | <b>0.091</b> | <b>0.956</b>   |
| Mixed           | 69.1% | 0.299 | 0.296 | 0.025          | <b>9.82%</b> | <b>0.007</b> | <b>0.034</b> | <b>0.974</b>   |

As expected, the estimates of the QT model are unacceptable in continuous-state traffic models, e.g. almost 70% for modulated exponentials. On the other hand, it achieves moderate accuracy for discrete-state models (Poisson, Deterministic, and On-Off). It is also noticeable how the QT model shows poor accuracy across all the jitter estimations. The main reason for this is that QT assumes independence between queues in the network. Hence, the estimator used to compute the jitter is the sum of the individual delay variance of queues along the flow's paths.

For non-Markovian traffic models (e.g., On-Off), RouteNet-F produces accurate estimates, as well as for more challenging models that implement strong autocorrelation (Autocorrelated Exponentials) and heavy-tail distributions (Modulated Exponentials). These models are important since they approximate real traffic generated by TCP [49], similar to that found at Internet links [13]. Also, notice that this traffic model could be made even more difficult for QT by increasing both the variance and the autocorrelation factor.

We add a final scenario (Mixed) where each src-dst pair generates traffic by randomly selecting one of the five available traffic models, and using random parameters for these models. In other words, we multiplex all traffic models in a single network topology. As the table shows, RouteNet-F shows good accuracy not only when modeling individual traffic models, but also when they are mixed across links in the network. It is worth noting that although RouteNet-F shows excellent accuracy for arbitrary parameterizations of these 6 traffic models, it cannot generalize to new traffic models not introduced during the training phase.

Finally, Table VII shows the different metrics for the packet loss ratio. Since there are some paths where the packet loss ratio is zero, we provide the Mean Absolute Error and the Coefficient of Determination ( $R^2$ ). The packet loss ratio is measured as the percentage of packets dropped w.r.t. packets sent, that is why the MAE is expressed in % units. In this particular case, it is noticeable how the QT baseline works well in the scenario with the Poisson traffic model. However, the accuracy decreases remarkably in more complex scenarios. On the other hand, RouteNet-F obtains a high accuracy with a worst-case MAE of 1.2% and  $R^2 \geq 0.98$ .

4) *Scheduling*: This section aims to validate if RouteNet-F is capable of modeling the behavior of queues in the presence of several scheduling policies. For this purpose, we train the model using samples with mixed queue scheduling policies across nodes in the GEANT and NSFNET topologies. Then, we evaluate the model on samples of the GBN topology (unseen during training). In this experiment, each router port implements three different queues with a randomly selected scheduling policy for the queues: (i) First In, First Out (FIFO), (ii) Weighted Fair Queueing (WFQ), (iii) Deficit Round Robin (DRR), and (iv) Strict Priority (SP). For WFQ and DRR, the set of weights is also randomly selected. Furthermore, each flow has been assigned a Quality-of-Service class that maps it to a specific queue depending on the flow priority. To provide a fair benchmark with QT, in this experiment we use only Poisson traffic.

Table VIII shows the Mean Average Percentage Error

TABLE VII: Packet Loss evaluation - Mean Absolute Error and Coefficient of Determination ( $R^2$ ) of QT and RouteNet-F for the different traffic models.

|            | Poisson     |             | Const. Bitrate |             | On-Off      |             | A. Exponentials |             | M. Exponentials |             | Multiplexed  |             |
|------------|-------------|-------------|----------------|-------------|-------------|-------------|-----------------|-------------|-----------------|-------------|--------------|-------------|
|            | MAE         | $R^2$       | MAE            | $R^2$       | MAE         | $R^2$       | MAE             | $R^2$       | MAE             | $R^2$       | MAE          | $R^2$       |
| QT         | 1.0%        | 0.97        | 11%            | 0.64        | 9.5%        | 0.63        | 10%             | 0.65        | 9.5%            | 0.08        | 4.6%         | 0.50        |
| RouteNet-F | <b>0.3%</b> | <b>0.99</b> | <b>1.0%</b>    | <b>0.99</b> | <b>1.0%</b> | <b>0.99</b> | <b>1.2%</b>     | <b>0.99</b> | <b>1.1%</b>     | <b>0.98</b> | <b>0.50%</b> | <b>0.99</b> |

TABLE VIII: Delay and jitter evaluation - Mean Absolute Percentage Error of QT and RouteNet-F in the presence of Scheduling Policies for low, medium, and high traffic intensity.

|            | Delay        |              |              | Jitter       |              |              |
|------------|--------------|--------------|--------------|--------------|--------------|--------------|
|            | Low          | Medium       | High         | Low          | Medium       | High         |
| QT         | 13.0%        | 17.3%        | 25.1%        | 49.0%        | 53.2%        | 59.6%        |
| RouteNet-F | <b>0.80%</b> | <b>2.60%</b> | <b>7.31%</b> | <b>3.95%</b> | <b>5.77%</b> | <b>14.8%</b> |

(MAPE) of the delay and jitter for three different traffic intensities: from low-loaded to highly-congested scenarios. According to [50] the average packet loss on the Internet is around 2%-3%. Based on this, in the highly-congested scenarios, the mean packet loss rate is around 3% which we believe represents a wide range of realistic network scenarios. We can see that RouteNet-F outperforms the QT benchmark in both metrics (delay and jitter), obtaining a MAPE of 3.57% for delay and 8.17% for jitter after averaging the results over the three traffic intensities.

Similarly, Table IX presents the results for packet loss. Again, RouteNet-F outperforms the QT benchmark showing an MAE of 0.7% and an average  $R^2$  close to 0.99.

TABLE IX: Packet loss evaluation - Mean Absolute Error and Coefficient of Determination ( $R^2$ ) of QT and RouteNet-F in the presence of Scheduling Policies for low, medium, and high traffic intensity.

|            | Low         |             | Medium       |             | High         |             |
|------------|-------------|-------------|--------------|-------------|--------------|-------------|
|            | MAE         | $R^2$       | MAE          | $R^2$       | MAE          | $R^2$       |
| QT         | 4.55%       | -0.05       | 9.22%        | 0.00        | 10.6%        | 0.29        |
| RouteNet-F | <b>0.2%</b> | <b>0.97</b> | <b>0.10%</b> | <b>0.99</b> | <b>0.40%</b> | <b>0.99</b> |

## B. Generalization and Scalability

1) *Generalization to larger networks*: The previous experiments show that RouteNet-F achieves remarkable accuracy when tested in scenarios with different traffic models (Section V-A3) and different scheduling policies (Section V-A4) with topologies never seen in training. As previously discussed in Section II, data-driven network models must generalize to larger networks than those seen during training to become a practical solution.

In this section, we evaluate RouteNet-F in a wide variety of networks significantly larger than the ones seen during the training phase. We generate a training set with 10,000 samples from networks of only 5 to 10 nodes. Following the process described in Sections IV-A and V-A4, for each flow, we randomly

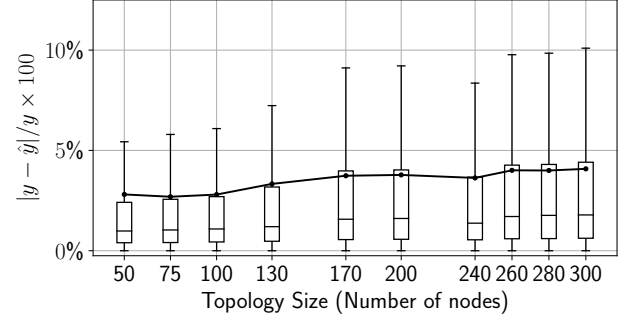


Fig. 7: Scaling with mixed traffic models and scheduling policies - Mean Absolute Relative Error of delay predictions vs. topology size, including different traffic models and queue scheduling configurations. The model was trained on a dataset with 10,000 samples from networks of 5 to 10 nodes.

assign a traffic model, and for each router port, we assign an arbitrary queue scheduling configuration. We evaluate the accuracy of RouteNet-F in topologies from 50 to 300 nodes, configuring the traffic models and the scheduling policies accordingly to the descriptions of Sections IV-A and V-A4. In contrast to previous experiments, all these networks have been synthetically generated using the Power-Law Out-Degree algorithm described in [27], where the ranges of  $\alpha$  and  $\beta$  parameters have been extrapolated from real-world topologies of the Internet Topology Zoo repository [51]. Link capacities and generated traffic volumes are scaled accordingly.

Figure 7 shows the MAPE of the delay predictions made by RouteNet-F. We can observe that the proposed model obtains a worst-case error of  $\approx 8\%$  for samples of networks with 300 nodes. This shows how RouteNet-F is capable of generalizing to networks 30x larger than those seen during training, even when introducing various traffic models and queue scheduling policies along the network. This is due to the capability of this model to effectively learn the underlying relationships between flows, links, and queues in the scenarios seen during training, and the posterior ability to exploit this learned knowledge in new scenarios not seen before.

Note that in this and the previous section, we have not tested any other baseline (e.g. RNN, QT) since they already fail in other *relevant* scenarios.

Despite generalization is an open challenge in the field of Deep Learning (as previously discussed in Section III) by using a custom GNN-based architecture and domain expert knowledge, RouteNet-F shows strong capabilities to generalize to considerably larger networks than the ones seen during training.

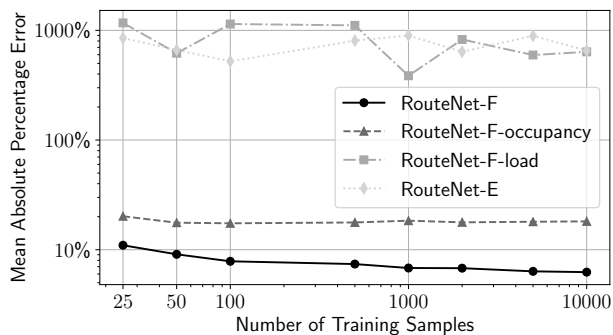


Fig. 8: Delay evaluation - Mean Absolute Percentage Error vs Number of Training Samples for the different versions of RouteNet-F and RouteNet-E.

2) *Few-shot Learning*: The performance of DL models is often determined by the quantity and quality of the training data used. Here, it is important to consider that the process of collecting and labeling large amounts of data can be very costly and, sometimes, infeasible. In the field of computer networks, such data collection implies generating and storing data from costly network infrastructures. This can be generally a very expensive and time-consuming process. An alternative is the use of packet-level simulators, which are also very expensive in terms of computational cost. In this section, we evaluate the accuracy of RouteNet-F when trained with a very limited number of samples (i.e., few-shot learning). This may be very helpful to dramatically reduce the cost of generating the datasets and reduce the carbon footprint of training the model.

To do so, we train the model by randomly selecting 25, 50, 100, 1,000, 2,000, 5,000, and 10,000 samples from the previous training dataset (Section V-B1). Note that the topologies used for training range from 5 to 10 nodes, while the evaluation is done over samples of networks from 50 to 300 nodes. Figure 8 shows the MAPE with respect to the number of training samples used (see only results of *RouteNet-F*). Interestingly, when trained with only 25 samples, the model shows an average error of 11%. As the number of samples increases, RouteNet-F obtains slightly better accuracy, achieving an error of 6.24% when trained with 10,000 samples.

3) *Ablation test*: We aim to analyze which features of RouteNet-F have more impact on its accuracy. For this purpose, we perform an ablation test, by considering four models where we remove different features. The first one (labeled as *RouteNet-F*) is the complete model used in the previous experiments, as it is previously described in Section IV. Second, in *RouteNet-F-occupancy* we remove the link load as an input feature (see Sec. IV-C1), and replace it directly with the link capacity value as an initial feature of links  $x_l$ . Third, *RouteNet-F-load* predicts the flow-level delay using directly the hidden state of flows ( $h_f$ ), instead of predicting the effective queue occupancy of flows at a specific link ( $h_{f,l}$ ) and then adding up all the estimated link-level delays (see Sec. IV-C2). Finally, we use *RouteNet-E* [43] as a reference, which is the previous version of RouteNet-F, without any of the aforementioned features. Figure 8 shows the results obtained in this experiment. We can see that predicting the

TABLE X: Inference time vs. topology size for RouteNet-F and the QT baseline.

|            | Topology Size |          |           |          |           |          |
|------------|---------------|----------|-----------|----------|-----------|----------|
|            | 10            | 30       | 50        | 70       | 90        | 110      |
| RouteNet-F | 48.03 ms      | 76.25 ms | 110.5 ms  | 285.6 ms | 455.3 ms  | 613.3 ms |
| QT         | 28.01 ms      | 49.19 ms | 131.68 ms | 326.5 ms | 662.15 ms | 962.5ms  |

delay as the sum of link-level delays along flows (*RouteNet-F-occupancy*) seems to have the largest impact on the accuracy of the model, achieving an error of 17.63%. In addition, the results suggest that using the link load as input instead of the capacity (*RouteNet-F-load*) does not have a significant impact on the model's accuracy. However, when we combine this feature with the one of *RouteNet-F-occupancy* we see a slight improvement. Particularly, in *RouteNet-F*, which implements the two features, we can see that the prediction error decreases to 6.24%.

In this experiment, it is observed that the previous version of the model (*RouteNet-E*) exhibits poor accuracy. This is likely because RouteNet-E was not designed to support scalability levels of 30x larger networks. Additionally, RouteNet-E assumes uniform delays for all flows traversing the same queue and link. In contrast, RouteNet-F takes into consideration that flows may experience different delays when traversing the same queue and link, depending on their traffic model (e.g., traffic burstiness).

4) *Scalability: Training and Inference time*: Models that are capable of producing fast estimations are particularly interesting for network control and management applications, as they can be deployed in real-time scenarios. In this section, we evaluate the inference time of both RouteNet-F and the QT baseline. To this end, we measure the inference times of the experiments of the general evaluation (Sec. V-B1). Table X shows that both models operate in the order of milliseconds for topologies lower than 110 nodes. Particularly, QT performs better for smaller topologies (<50 nodes). However, as the size of the topology increases, RouteNet-F is faster.

Finally, note that one difference between analytical models (e.g., QT) and DL-based models (e.g., RouteNet-F) is that the last need a training process that may be taken into consideration when comparing these times. These training times depend greatly on various factors like the size of the training dataset, the hyperparameters, the used hardware, etc. In our particular case, the model that obtained the higher accuracy was trained during 20 epochs of 2,500 samples each, lasting for about 2h30min.

In this experiment, we used one CPU [AMD Ryzen 9 3950X @ 3.5 GHz]. However, an advantage of DL-based models is that they can be easily parallelizable using hardware-specific solutions (e.g., GPU), thus reducing considerably their execution times in production.

### C. Benchmarking of RouteNet-F

1) *Testbed*: In previous sections, we examined how RouteNet-F can be applied in different network scenarios. This section evaluates its performance in a real-world scenario using real-world hardware and synthetic traffic. For this, we set up a physical network testbed, as shown in Figure 9. This



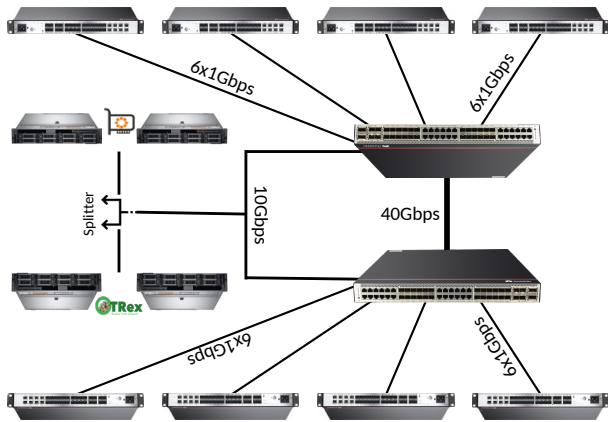


Fig. 9: Schematic representation of the network testbed.

testbed includes (i) 8 Huawei NetEngine 8000 M1A routers, (ii) 2 Huawei S5732-H48UM 2CC 5G Bundle switches, and (iii) 4 servers. Two of the servers are used to generate traffic using the TRex traffic generator, and the other two are used for capturing and analyzing traffic with the PF\_RING software.

To train and test the model, we generated 1,000 samples with realistic network topologies (with a maximum of 8 nodes) and different routing, queueing, and traffic configurations. Out of these 1,000 samples, we randomly selected 800 samples for training the model and used the remaining 200 samples for testing. Note that, the algorithm 1 described in Section IV-A has been slightly modified to add the state of the queues of the switches found in the topology. This only affects the initialization process, and not the message-passing architecture itself.

Table XI shows the results of RouteNet-F. As can be seen, RouteNet-F obtains a remarkable performance (11% MAPE) which is in line with the previous results when compared with simulated data.

2) *Real Traffic*: In our previous experiments, we tested RouteNet-F using synthetic-simulated traffic. Now, we want to see how well this model performs when applied to actual traffic data.

For this, we used real-world traffic data from the SNDlib library [52] and combined it with packet inter-arrival times from a recent snapshot of the MAWI repository (Sample point 2022/09) [53]. We then scale the inter-arrival times to match the values in the traffic data. Additionally, we used a distribution of source-destination flows from a real internet service provider [54] to map flows to different ToS classes. Our dataset includes 4 real-world network topologies, including one previously used (GEANT) and three new topologies that the model has never seen before (ABILENE, NOBEL-GBN, and GERMANY50). For this experiment, we leveraged the knowledge obtained from previous versions and used a previous checkpoint, fine-tuning it using 200 samples of the GEANT topology.

Table XI shows the results of RouteNet-F. We can see that RouteNet-F achieves a remarkable performance (5.67% MAPE) when tested using real-world traffic data. Again, these results are close to the simulator and testbed ones.

3) *State-of-the-Art*: This section aims to compare the accuracy of RouteNet-F against MimicNet [16]. MimicNet is a DL-

TABLE XI: Delay prediction using RouteNet-F for the testbed and the real traffic traces experiments.

|              | MAPE  | MSE                   | MAE    | R <sup>2</sup> |
|--------------|-------|-----------------------|--------|----------------|
| Testbed      | 11.0% | $6.12 \times 10^{-5}$ | 0.0007 | 0.869          |
| Real Traffic | 5.67% | $1.66 \times 10^{-5}$ | 0.0003 | 0.877          |

based model which combines discrete event simulators with Deep Neural Networks (DNN). MimicNet takes advantage of the accuracy of discrete packet-event simulators to generate data of a small network which then is used to train an RNN-based estimator known as a "mimic". Finally, MimicNet composes several of those mimics to perform predictions of much bigger larger networks. This makes MimicNet an alternative to a discrete packet-event simulator as it reduces the cost of simulation for large networks, providing an accurate estimation of the per-packet level distributions. However, as stated in the MimicNet paper, the main limitation of this strategy is that it only works for FatTree topologies.

Following the parameters and the specifications described in [17] we generate a dataset containing three different topologies (FatTree16, FatTree64, and FatTree128) and compute the average RTT.

Table XII shows the results for both RouteNet-F and MimicNet. For RouteNet-F we show the same metrics as before and the Normalized Wasserstein Distance ( $W_1$ ). Contrary to RouteNet, MimicNet does not directly compare the average RTT of the packets aggregated per path. Instead, it computes the distance of both distributions (the predicted and the real one).

Note that in this scenario, RouteNet-F achieves an outstanding accuracy not only when compared with MimicNet, but also when compared with a state-of-the-art QT model (Table V). This is mainly because, in previous experiments, we explored a wide variety of scenarios containing from low to high (3%) packet loss ratios, while in this particular scenario, the packet loss rate is about 0.3%.

Finally, as shown in section V-B4, RouteNet-F is capable of predicting the performance metrics in the order of milliseconds. In contrast, MimicNet ranges from minutes for the smaller topologies to hours for the larger ones. This difference is mainly because, while RouteNet-F focuses on predicting the different performance metrics aggregated by flows, MimicNet predicts those metrics at a packet level.

TABLE XII: Average RTT prediction using MimicNet and RouteNet-F for different FatTree topologies. The error is computed w.r.t. simulation results.

|            | MimicNet | RouteNet-F   |                                          |                                         |                |               |
|------------|----------|--------------|------------------------------------------|-----------------------------------------|----------------|---------------|
|            | $W_1$    | MAPE         | MSE                                      | MAE                                     | R <sup>2</sup> | $W_1$         |
| FatTree16  | 0.0080   | <b>0.37%</b> | <b><math>1.31 \times 10^{-10}</math></b> | <b><math>5.33 \times 10^{-6}</math></b> | <b>0.999</b>   | <b>0.0018</b> |
| FatTree64  | 0.0122   | <b>0.44%</b> | <b><math>1.70 \times 10^{-10}</math></b> | <b><math>7.15 \times 10^{-6}</math></b> | <b>0.999</b>   | <b>0.0026</b> |
| FatTree128 | 0.0172   | <b>0.67%</b> | <b><math>3.40 \times 10^{-10}</math></b> | <b><math>1.20 \times 10^{-5}</math></b> | <b>0.998</b>   | <b>0.0060</b> |

## VI. RELATED WORK

The use of Deep Learning (DL) for network modeling has recently attracted much interest from the networking community. The authors from [31] survey different techniques and

discuss data-driven models that can learn from real networks. Initial attempts to implement this idea use fully-connected neural networks (e.g., [30], [55]). Such early attempts do not generalize to different networks unseen in training, they are not tested with realistic traffic models, and they do not model QoS-aware queue scheduling configurations. More recent works propose elaborated neural networks models, like Variational Auto-encoders [56] or ConvNN [57]. However, they have similar limitations.

Since the introduction of Graph Neural Networks [37], they have already been applied to different fields such as chemistry [14] or logistics [15]. In the field of computer networks, early pioneering works leverage GNNs [58], [59]. However, they use a basic GNN architecture that considers a simplified model of the network, ignoring traffic models, queuing policies, and the critical property of generalizing to larger networks.

More recent works leverage GNNs to model complex characteristics of computer networks. For example, xNet [60] learns the state transition function between time steps and rolls it out to obtain the complete fine-grained prediction trajectory, but it does not take into account scaling to larger networks. Others focus on data center networks to predict the Flow Completion Time [61]. Finally, state-of-the-art simulators MimicNet [16] and DeepQueueNet [17] leverage DL models (LSTMs and transformers, respectively) to accelerate parts of the simulation process, with a focus on per-packet delay prediction, as opposed to our per-path average delay prediction.

## VII. CONCLUSION

In this paper, we have presented RouteNet-Fermi, a custom GNN model designed for network performance analysis. This model supports a wide range of configuration parameters related to routing, queue scheduling, and traffic models while being able to accurately model networks 30 times larger than the ones seen during training. In our evaluation, we have shown that the proposed model outperforms a state-of-the-art queuing theory model, especially in scenarios with complex and realistic traffic models. At the same time, RouteNet-Fermi achieves comparable accuracy with respect to computationally-expensive packet-level simulators (MAPE  $\approx$  6.24%) while exhibiting considerably lower inference times (on the order of milliseconds in networks of 100 nodes). Finally, we validated RouteNet-F in a wide variety of real-world scenarios including a testbed and real-world traffic traces.

## REFERENCES

- [1] L. Kleinrock, "Analytic and simulation methods in computer network design," in *Proceedings of the May 5-7, 1970, spring joint computer conference*, 1970, pp. 569–579.
- [2] T. G. Robertazzi, *Computer networks and systems: queueing theory and performance evaluation*. Springer Science & Business Media, 2000.
- [3] R. M. Fujimoto *et al.*, "Large-scale network simulation: how big? how fast?" in *11th IEEE/ACM International Symposium on Modeling, Analysis and Simulation of Computer Telecommunications Systems, 2003. MASCOTS 2003*. IEEE, 2003, pp. 116–123.
- [4] P. Almasan, M. Ferriol-Galmés *et al.*, "Digital twin network: Opportunities and challenges," *arXiv preprint arXiv:2201.01144*, 2022.
- [5] H. Nguyen *et al.*, "Digital twin for 5g and beyond," *IEEE Communications Magazine*, vol. 59, no. 2, pp. 10–15, 2021.
- [6] S. Mirjalili, "Genetic algorithm," in *Evolutionary algorithms and neural networks*. Springer, 2019, pp. 43–55.
- [7] Z. Xu *et al.*, "Experience-driven networking: A deep reinforcement learning based approach," in *IEEE INFOCOM*, 2018, pp. 1871–1879.
- [8] A. Arfeen, K. Pawlikowski *et al.*, "The role of the weibull distribution in modelling traffic in internet access and backbone core networks," *Journal of network and computer applications*, vol. 141, pp. 1–22, 2019.
- [9] T. Karagiannis *et al.*, "A nonstationary poisson view of internet traffic," in *IEEE INFOCOM*, vol. 3, 2004, pp. 1558–1569.
- [10] T. Karagiannis, M. Molle, and M. Faloutsos, "Long-range dependence ten years of internet traffic modeling," *IEEE internet computing*, vol. 8, no. 5, pp. 57–64, 2004.
- [11] E. Kresch and S. Kulkarni, "A poisson based bursty model of internet traffic," in *IEEE International Conference on Computer and Information Technology*, 2011, pp. 255–260.
- [12] V. Paxson and S. Floyd, "Wide area traffic: the failure of poisson modeling," *IEEE/ACM Transactions on networking*, vol. 3, no. 3, pp. 226–244, 1995.
- [13] J. Popoola and R. Ipinyomi, "Empirical performance of weibull self-similar tele-traffic model," *International Journal of Engineering and Applied Sciences*, vol. 4, no. 8, p. 257389, 2017.
- [14] J. Gilmer *et al.*, "Neural message passing for quantum chemistry," in *International conference on machine learning*. PMLR, 2017, pp. 1263–1272.
- [15] E. E. Kosasih and A. Brintrup, "A machine learning approach for predicting hidden links in supply chain with graph neural networks," *International Journal of Production Research*, pp. 1–14, 2021.
- [16] Q. Zhang *et al.*, "Mimicnet: fast performance estimates for data center networks with machine learning," in *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, 2021, pp. 287–304.
- [17] Q. Yang, X. Peng *et al.*, "DeepqueueNet: Towards scalable and generalized network performance estimation with packet-level visibility," in *Proceedings of the ACM SIGCOMM 2022 Conference*, ser. SIGCOMM '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 441–457. [Online]. Available: <https://doi.org/10.1145/3544216.3544248>
- [18] Q. Rao and J. Frtunikj, "Deep learning for self-driving cars: Chances and challenges," in *Proceedings of the 1st international workshop on software engineering for AI in autonomous systems*, 2018, pp. 35–38.
- [19] M. Ferriol-Galmés *et al.*, "Routenet-fermi," [https://github.com/BNN-UPC/Papers/wiki/RouteNet\\_Fermi](https://github.com/BNN-UPC/Papers/wiki/RouteNet_Fermi), 2022.
- [20] J. Ma, J. Deng, and Q. Mei, "Subgroup generalization and fairness of graph neural networks," *Advances in Neural Information Processing Systems*, vol. 34, pp. 1048–1061, 2021.
- [21] G. Yehudai, E. Fetaya, E. Meirum, G. Chechik, and H. Maron, "From local structures to size generalization in graph neural networks," in *International Conference on Machine Learning*. PMLR, 2021, pp. 11 975–11 986.
- [22] A. Al-Sawaai, I. Awan, and R. Fretwell, "Performance evaluation of weighted fair queuing system using matrix geometric method," in *International Conference on Research in Networking*. Springer, 2009, pp. 66–78.
- [23] A. S. Kapadia, M. F. Kazmi, and A. C. Mitchell, "Analysis of a finite capacity non preemptive priority queue," *Computers & operations research*, vol. 11, no. 3, pp. 337–343, 1984.
- [24] E. Weingartner, H. Vom Lehn, and K. Wehrle, "A performance comparison of recent network simulators," in *2009 IEEE International Conference on Communications*. IEEE, 2009, pp. 1–5.
- [25] A. Varga, "Discrete event simulation system," in *European Simulation Multiconference (ESM)*, 2001, pp. 1–7.
- [26] G. F. Riley and T. R. Henderson, "The ns-3 network simulator," in *Modeling and tools for network simulation*. Springer, 2010, pp. 15–34.
- [27] C. R. Palmer and J. G. Steffan, "Generating network topologies that obey power laws," in *Globecom'00-IEEE. Global Telecommunications Conference. Conference Record (Cat. No. 00CH37137)*, vol. 1. IEEE, 2000, pp. 434–438.
- [28] S. K. Pal and S. Mitra, "Multilayer perceptron, fuzzy sets, and classification," *IEEE Transactions on neural networks*, vol. 3, no. 5, pp. 683–697, 1992.
- [29] N. Sadeghzadeh *et al.*, "An mlp neural network for time delay prediction in networked control systems," in *2008 Chinese Control and Decision Conference*. IEEE, 2008, pp. 5314–5318.
- [30] A. Mestres *et al.*, "Understanding the modeling of computer network delays using neural networks," in *Proceedings of the 2018 Workshop*

- on Big Data Analytics and Machine Learning for Data Communication Networks, 2018, pp. 46–52.
- [31] M. Wang *et al.*, “Machine learning for networking: Workflow, advances and opportunities,” *Ieee Network*, vol. 32, no. 2, pp. 92–99, 2017.
  - [32] T. Mikolov, S. Kombrink *et al.*, “Extensions of recurrent neural network language model,” in *2011 IEEE international conference on acoustics, speech and signal processing (ICASSP)*. IEEE, 2011, pp. 5528–5531.
  - [33] S. Belhaj and M. Tagina, “Modeling and prediction of the internet end-to-end delay using recurrent neural networks,” *J. Networks*, vol. 4, no. 6, pp. 528–535, 2009.
  - [34] A. R. Mohammed *et al.*, “Machine learning and deep learning based traffic classification and prediction in software defined networking,” in *2019 IEEE International Symposium on Measurements & Networking (M&N)*. IEEE, 2019, pp. 1–6.
  - [35] S. Naseer *et al.*, “Enhanced network anomaly detection based on deep neural networks,” *IEEE access*, vol. 6, pp. 48 231–48 246, 2018.
  - [36] J. Suárez-Varela, P. Almasan, M. Ferriol-Galmés *et al.*, “Graph neural networks for communication networks: Context, use cases and opportunities,” *IEEE Network*, 2022.
  - [37] F. Scarselli, M. Gori *et al.*, “The graph neural network model,” *IEEE transactions on neural networks*, vol. 20, no. 1, pp. 61–80, 2008.
  - [38] P. W. Battaglia *et al.*, “Relational inductive biases, deep learning, and graph networks,” *arXiv preprint arXiv:1806.01261*, 2018.
  - [39] J. Zhou *et al.*, “Graph neural networks: A review of methods and applications,” *arXiv preprint arXiv:1812.08434*, 2018.
  - [40] L. Ruiz, L. Chamon, and A. Ribeiro, “Graphon neural networks and the transferability of graph neural networks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 1702–1712, 2020.
  - [41] L. Engstrom, B. Tran *et al.*, “Exploring the landscape of spatial robustness,” in *International Conference on Machine Learning*, 2019, pp. 1802–1811.
  - [42] J. Su, D. V. Vargas, and K. Sakurai, “One pixel attack for fooling deep neural networks,” *IEEE Transactions on Evolutionary Computation*, vol. 23, no. 5, pp. 828–841, 2019.
  - [43] M. Ferriol-Galmés *et al.*, “Routenet-erlang: A graph neural network for network performance evaluation,” in *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 2022, pp. 2018–2027.
  - [44] K. Rusek, “Queuinx,” <https://github.com/krzysztofusek/queuinx>, 2023.
  - [45] M. F.-G. Albert López *et al.*, “Bnnetsimulator,” 2023. [Online]. Available: <https://github.com/BNN-UPC/BNNetsimulator>
  - [46] X. Hei, J. Zhang *et al.*, “Wavelength converter placement in least-load-routing-based optical networks using genetic algorithms,” *Journal of Optical Networking*, vol. 3, no. 5, pp. 363–378, 2004.
  - [47] F. Barreto *et al.*, “Fast emergency paths schema to overcome transient link failures in ospf routing,” *arXiv preprint arXiv:1204.2465*, 2012.
  - [48] J. Pedro, J. Santos, and J. Pires, “Performance evaluation of integrated otn/dwdm networks with single-stage multiplexing of optical channel data units,” in *International Conference on Transparent Optical Networks*, 2011, pp. 1–4.
  - [49] D. R. Figueiredo, B. Liu, V. Misra, and D. Towsley, “On the autocorrelation structure of tcp traffic,” *Computer Networks*, vol. 40, no. 3, pp. 339–361, 2002.
  - [50] S. Floyd and V. Paxson, “Difficulties in simulating the internet,” *IEEE/ACM Transactions on Networking*, vol. 9, no. 4, pp. 392–403, 2001.
  - [51] S. Knight *et al.*, “The internet topology zoo,” *IEEE Journal on Selected Areas in Communications*, vol. 29, no. 9, pp. 1765–1775, 2011.
  - [52] S. Orlowski, R. Wessäly *et al.*, “Sndlib 1.0—survivable network design library,” *Networks: An International Journal*, vol. 55, no. 3, pp. 276–286, 2010. [Online]. Available: <http://sndlib.zib.de>
  - [53] R. Fontugne, P. Borgnat *et al.*, “MAWILab: Combining Diverse Anomaly Detectors for Automated Anomaly Labeling and Performance Benchmarking,” in *ACM CoNEXT '10*, Philadelphia, PA, December 2010, pp. –.
  - [54] S. Schnitter *et al.*, “Quality-of-service class specific traffic matrices in ip/mppls networks,” in *ACM Internet Measurement Conference*, 2007, pp. 253–258.
  - [55] A. Valadarsky *et al.*, “Learning to route,” in *ACM workshop on hot topics in networks*, 2017, pp. 185–191.
  - [56] S. Xiao *et al.*, “Deep-q: Traffic-driven qos inference using deep generative network,” in *ACM SIGCOMM Workshop on Network Meets AI & ML*, 2018, pp. 67–73.
  - [57] X. Chen *et al.*, “Deep-rmsa: A deep-reinforcement-learning routing, modulation and spectrum assignment agent for elastic optical networks,” in *2018 Optical Fiber Communications Conference and Exposition (OFC)*, 2018, pp. 1–3.
  - [58] F. Geyer and S. Bondorf, “Deeptma: Predicting effective contention models for network calculus using graph neural networks,” in *IEEE INFOCOM*, 2019, pp. 1009–1017.
  - [59] K. Rusek, J. Suárez-Varela *et al.*, “Unveiling the potential of graph neural networks for network modeling and optimization in sdn,” in *ACM Symposium on SDN Research*, 2019, pp. 140–151.
  - [60] M. Wang, L. Hui, Y. Cui *et al.*, “xnnet: Improving expressiveness and granularity for network modeling with graph neural networks,” in *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 2022, pp. 2028–2037.
  - [61] J. Li, P. Sun, and Y. Hu, “Traffic modeling and optimization in datacenters with graph neural network,” *Computer Networks*, vol. 181, p. 107528, 2020.